

EXPERIMENT 8

Aim: To implement task assignment approach

Objective: Implement task assignment approach

Theory:

In task assignment approach, process is viewed as a collection of related tasks and these tasks are scheduled to suitable nodes so as to improve performance.

A process is split into pieces called tasks. This split occurs along natural boundaries, so that data transfers among the tasks is minimized. The amount of computation required by each task and the speed of each processor are known. The cost of processing each task on every node of the system is also known. The inter-process communication (IPC) costs between every pair of tasks are known.

Other constraints, such as resource requirements of the tasks and the available resources at each node, precedence relationships among the tasks, and so on, are also known.

Consider the following example:

Intertask communication costs						
	t_1	t_2	t_3	t_4	t_5	t_6
t_1	0	6	4	0	0	12
t_2	6	0	8	12	3	0
t_3	4	8	0	0	11	0
t_4	0	12	0	0	5	0
t_5	0	3	11	5	0	0
t_6	12	0	0	0	0	0

(a)

Execution costs		
Tasks	Nodes	
	n_1	n_2
t_1	5	10
t_2	2	∞
t_3	4	4
t_4	6	3
t_5	5	2
t_6	∞	4

(b)

The serial assignment cost can be calculated as follows:

$$\begin{aligned} \text{Serial assignment execution cost } (x) &= x_{11} + x_{21} + x_{31} + x_{42} + x_{52} + x_{62} \\ &= 5 + 2 + 4 + 3 + 2 + 4 = 20 \end{aligned}$$

$$\begin{aligned} \text{Serial assignment communication cost } (c) &= c_{14} + c_{15} + c_{16} + c_{24} + c_{25} + c_{26} + c_{34} + c_{35} + c_{36} \\ &= 0 + 0 + 12 + 12 + 3 + 0 + 0 + 11 + 0 = 38 \end{aligned}$$

$$\text{Serial assignment total cost} = x + c = 20 + 38 = 58$$

And the optimal assignment cost can be calculated as follows:

$$\begin{aligned} \text{Optimal assignment execution cost } (x) &= x_{11} + x_{21} + x_{31} + x_{41} + x_{51} + x_{62} \\ &= 5 + 2 + 4 + 6 + 5 + 4 = 26 \end{aligned}$$

$$\begin{aligned} \text{Optimal assignment communication cost } (c) &= c_{16} + c_{26} + c_{36} + c_{46} + c_{56} \\ &= 12 + 0 + 0 + 0 + 0 = 12 \end{aligned}$$

$$\text{Optimal assignment total cost} = x + c = 26 + 12 = 38$$

For implementing task assignment approach, take from the user all the parameters and then calculate the cost.

Code and output:

class RoundRobinLoadBalancer:

```
def __init__(self, servers): # Corrected __init__
    # Initialize list of servers and set current server index to 0
    self.servers = servers
    self.index = 0

def get_next_server(self):
```

```

    # Get the server assigned to handle the current request
    server = self.servers[self.index]

    # Update the index to point to the next server
    self.index = (self.index + 1) % len(self.servers) # Loop back if the index exceeds the number of
servers

    return server

def handle_request(self, request):

    # Get the server that will handle the current request
    server = self.get_next_server()

    # Server processes the request (simulated here with a print statement)
    print(f"Request {request} is assigned to {server}")

# List of servers (or nodes)
servers = ['Server1', 'Server2', 'Server3']

# Initialize the Round Robin Load Balancer with the list of servers
load_balancer = RoundRobinLoadBalancer(servers)

# Simulate handling 5 requests
for request_id in range(1, 6):
    load_balancer.handle_request(request_id)

```

```

Request 1 is assigned to Server1
Request 2 is assigned to Server2
Request 3 is assigned to Server3
Request 4 is assigned to Server1
Request 5 is assigned to Server2

```

Conclusion:

The execution time of the Round Robin Load Balancer depends on the number of requests (**N**) and the number of servers (**M**). Each request requires $O(1)$ time to fetch the next server and assign the request. The total assignment time for **N** requests is $O(N)$, as each request is processed in constant time regardless of the number of servers.